Assignment 06

Name: Kaifeng Tang ID: 524030910124

June 12, 2026

Problem 1

(a)

Transfer to Network: Build a bipartite graph $G = (L, R, E)$.

- Source node: s; Sink node: t.
- $L = \{a_1, a_2, \ldots, a_k\}$, where a_i represents subset $\mathcal{A}_i$. $R = U = \{1, 2, \ldots, n\}$.
- Capacity Constraint: $c(e) = 1$ for $\forall e \in E$.

Algorithm:

Function judgeSDR(U, $\mathcal{A}$):

Build flow network $G = (A, U, E)$ by rules mentioned above.
Run Fax-Flow Algorithm in G
get the max-flow F
if $F = k$:
    return $true$
else return $false$

Proof of Correctness:

We need to prove bi-directional corresponding:

$$\mathcal{A} \text{ has a system of distinct representatives} \iff G\text{exists max-flow } F$$

(i) Sufficiency: assume $F = k$. Since all edges (s, a_i) have capacity 1, and there are k edges starting from s.

$\implies$ every edge (a, a_i) holds $f(e) = c(e)$, which means full flow.

$\implies$ each a_i has a flow to some u_j.

Since (a_i, u_j) has capacity only 1, so different a_i must push flow to distinct u_j. With capacity 1 in (u_j, t), all flows can sink to t.

$\therefore$ one u_j at most receive flow $f = 1$ from one a_i. All these u_j forms a system of distinct representatives.

(ii) Necessity: assume $\mathcal{A}$ exists a system of distinct representatives. So there exists distinct $u_j \in U$ s.t. $u_{j_i} \in \mathcal{A}_i$. So the edges (a_i, u_j) exists.

We can create a flow: $s \to a_i \to u_j \to t$, push single flow 1 on the path. By the capacity constraint, the flow is legal. The total flow: k.

On the other hand, the number of edges from s equals to k, indicating the max-flow is also k.

By Intgrality Theorem, integral capacity $\to$ integral flow. So we have

$$\mathcal{A} \text{ has a system of distinct representatives} \iff G\text{exists max-flow } F$$

Time Complexity: Let $m_A = \sum_{i=1}^{k} |\mathcal{A}_i|$. So $|V| = k + n + 2$; $|E| = m_A + k + n$.

By Ford-Fulkerson Algorithm: flow added 1 by one iteration $\rightarrow$ at most k iteration times. Every time find a path: $O(|E|)$ time.

$\Longrightarrow$ Overall time: $O(k|E|)$. If using Dinic Algorithm: $O(|V|^2|E|)$, HKK Algorithm: $O(|E|\sqrt{|V|})$.

(b)

Transfer to Network: Build $G' = (V', E')$: with flow path $s \rightarrow \mathcal{A}_i \rightarrow \mathcal{B}_j \rightarrow t$.

- Source node: s; Sink node: t.
- u means it is the representatives of both an $\mathcal{A}_i$ and an $\mathcal{B}_j$.
- Capacity Constraint: $c(e) = 1$ for $\forall e \in E$.

Algorithm:

Function judgeCommonSDR(U, $\mathcal{A}$, $\mathcal{B}$):

> Build flow network $G' = (V', E')$ by rules mentioned above.
> Run Fax-Flow Algorithm in G'
> get the max-flow F
> if $F = k$:
> return $true$
> else return $false$

Proof of Correctness:

Claim: $\mathcal{A}$ and $\mathcal{B}$ has a system of distinct representatives $\iff$ Gexists max-flow F

(i) Necessity: assume $\mathcal{A}$ and $\mathcal{B}$ exists a common system of distinct representatives $T = \{u_1, u_2, \ldots, u_k\}$. For $u \in T$: u can represent some $a_i \in \mathcal{A}_i$, $b_j \in \mathcal{B}_j$.

If we push $f = 1$ on a path: $s \rightarrow a_i \rightarrow u \rightarrow b_j \rightarrow t$, by the rules of G', all the edges exist. By capacity constraint: $f \leq c$, consider a node $u \in T$: $u_{in} = u_{out} \leq 1$. So the flow is legal. Since the number of edges $(s, a_i) = k$, the max-flow is k.

(ii) Sufficiency: assume $F = k$. Since all edges (s, a_i) have capacity 1, and there are k edges starting from s, so every edge (a, a_i) holds $f(e) = c(e)$, which means full flow.

For every $\mathcal{A}_i$: exists some u: $s \rightarrow a \rightarrow u$. Then u can be the representative of $\mathcal{A}$.

For every $\mathcal{B}_j$: The same. So the same element u can be used by at most one unit flow path.

$\therefore$ all the u_j forms a common system of distinct representatives.

By Intgrality Theorem, integral capacity $\rightarrow$ integral flow. So we have

$$\mathcal{A} \text{ and } \mathcal{B} \text{ has a system of distinct representatives} \iff G\text{exists max-flow } F$$

Time Complexity: Let $m_A = \sum_{i=1}^{k} |\mathcal{A}_i|$, $m_B = \sum_{i=1}^{k} |\mathcal{B}_i|$. So $|V'| = k + 2n + 2$; $|E'| = m_A + m_B + 2k + n$.

The same as Problem (a): Overall time: $O(k|E'|)$.

If using Dinic Algorithm: $O(|V'|^2|E'|)$.

HKK Algorithm: $O(|E'|\sqrt{|V'|})$.

Problem 2

(a)

By Dinic's Algorithm: we know that each iteration needs $O(|E|)$ time to find a blocking flow and updating G^f. Thus, to prove Dinic's Algorithm runs in $O(|E|^{3/2})$ time, we just need to prove Dinic's number of iterations is at most $O(|E|^{1/2})$ time.

Proof. Consider the Algorithm just run $\sqrt{|E|}$ iterations.

By Property of Dinic's Algorithm: $dist_{i+1}(u) > dist_i(u)$ after one iteration, we can get: After $\sqrt{|E|}$ rounds, the distance of all s-t paths in G^f are $\geq \sqrt{|E|}$.

Let f = Current flow, f^* = max-flow. Consider rest of $|f^*| - |f|$ flows.

These flows can be decomposed into edge-disjoint s-t paths. Each such path has a length of at least $\sqrt{|E|}$. Since the number of residual edges in G^f is $O(|E|)$.

$\implies$ The maximum number of paths with non-intersecting edges: $\frac{O(|E|)}{\sqrt{|E|}} = O(\sqrt{|E|})$.

□

(b)

The same as Problem (a): we know each iteration needs $O(|E|)$ time, so we just need to prove Dinic's number of iterations is at most $O(|V|^{2/3})$ time.

Proof. Consider Dinic's Algorithm runs $2|V|^{2/3}$ rounds. Suppose Algorithm doesn't halts. Suppose that after $2|V|^{2/3}$ rounds, the current flow is f.

Definition: $D_i = \{v \in V : dist_{G^f}(s, v) = i\}$ in G^f. Since after each iteration, the shortest distance s-t strictly increases.

Definition: $\ell = dist_{G^f}(s, t)$

So we have $\ell \geq 2|V|^{2/3}$.

Now prove $|D_i \cup D_{i+1}| \leq |V|^{1/3}$:

Since each node appears in at most two such unions $D_i \cup D_{i+1}$

$\implies \sum_{i=0}^{\ell-1} |D_i \cup D_{i+1}| \leq 2|V|$, by $\ell \geq 2|V|^{2/3} \implies$ there exists some i s.t. $|D_i \cup D_{i+1}| \leq \frac{2|V|}{\ell} \leq \frac{2|V|}{2|V|^{2/3}} = |V|^{1/3}$.

Since $dist(v) \leq dist(u) + 1$ for (u, v), so it is necessary to use a residual edge from $D_i \to D_{i+1}$. The maximum number of possible edges from $D_i \to D_{i+1}$:

$$|D_i| \cdot |D_{i+1}| \leq |D_i \cup D_{i+1}|^2 \leq |V|^{2/3}$$

$\implies$ The number of iterations is at most $O(|V|^{2/3})$. □

Problem 3

(a)

The maximum matching problem can be written as an IP problem first:

$$\begin{aligned} \text{maximize} \quad & \sum_{e \in E} x_e \\ \text{subject to} \quad & \sum_{e : e \in (u,v)} x_e \leq 1 && (\forall v \in V) \\ & x_e \in \{0, 1\} && (\forall e \in E) \end{aligned}$$

Objective: represents the number of selected edges e, so maximizing $\sum_{e \in E} x_e$ is equivalent to finding the maximum matching.

Constraints: $\sum_{e:e\in(u,v)} x_e \leq 1$ means each vertex can be covered by at most one selected edge, which equals to the definition of matching.

$$x_e = \begin{cases} 1, & \text{if edge e is chosen in the matching,} \\ 0, & \text{otherwise} \end{cases}$$

In problem (a), constraint 2 is relaxed to $x_e \geq 0$. Since any 0/1 solution of a matching satisfies the LP constraint in the problem, a feasible solution of a matching must be a feasible solution of the LP.

$\therefore$ This LP is a relaxation of maximum matching problem.

(b)

For the vertex constraint: $\sum_{e:e\in(u,v)} x_e \leq 1$, introduce a dual variable $y_v \geq 0$ for every vertex v.

The dual objective: min $\sum_{v\in V} y_v$, since the right side is 1 for primal constraint.

Dual constraints: $y_v + y_u \geq 1, \quad \forall e \in E$, since the coefficient of primal objective is 1. The constraint holds for every edges (u, v).

The Dual LP:

$$\begin{aligned} \text{minimize} \quad & \sum_{v\in V} y_v & \\ \text{subject to} \quad & y_u + y_v \geq 1 & (\forall e \in E) \\ & y_v \geq 0 & (\forall v \in V) \end{aligned}$$

Explanation: The dual objective $\sum_{v\in V} y_v$ choose as few vertices as possible, meeting the definition of *Minimum Vertex Cover*.

Constraint: $y_u + y_v \geq 1$ means that for each edge $e = (u, v)$, t least one endpoint of u or v must be selected, so that the edge is covered.

In problem *Minimum Vertex Cover*: $y_v = \begin{cases} 1, & v \text{ is chosen,} \\ 0, & \text{otherwise} \end{cases}$ For dual constraint $y_v \geq 0$, it is a relaxation of IP constraint $y_v \in \{0, 1\}$.

$\therefore$ The dual LP is a relaxation of Problem *Minimum Vertex Cover*.

(c)

Let $G = (L, R, E)$. By bipartite, all $e \in E$ connect a vertex in L and a vertex in R; no edge inside L or R. By definition of *incident matrix*, each column corresponds to one edge in A. $\implies$ each column has at most two "1"s.

Take any $q \times q$ submatrix B of A, we want to prove $\det(B) \in (-1, 0, 1)$ by induction.

Base. When $q = 1$, B has only one element. $\det(B) \in (0, 1)$, correct.

Induction Hypothesis. Suppose for all $r \times r$ $(r = 1, 2, \ldots, q-1)$ submatrix B', $\det(B) \in (-1, 0, 1)$holds.

Induction. There are only 3 possible cases:

- *Case 1.* There exists a column in B with no $0 \to \det(B) = 0$, correct.
- *Case 2.* A column has exactly one non-zero element, which is exactly $1 \to$ By expanding the determinant along this column: $\det(B) = \pm\det(B')$, where B' is an incident submatrix with size $(q-1) \times (q-1)$. We can easily get $\det(B) \in (-1, 0, 1)$, correct.

- *Case 3.* Each column in B contains exactly two 1s. $\rightarrow$ These two 1s must appear in the row corresponding to L and the row corresponding to R, respectively.
  The entire row vector satisfies: $\sum_{\text{row}\in L}\text{row} = \sum_{\text{row}\in R}\text{row}$, which indicates row vectors of B are linearly dependent.
  So $\det(B) = 0$, correct.

By conclude, $\det(B) \in (-1, 0, 1)$ holds for all submatrix $B \subseteq A$.
$\implies$ the *incident matrix* of a bipartite graph is totally unimodular. □

(d)

From 3(b), we can get dual LP of maximum matching. By 3(c), turn LP to matrix form:

$$\begin{aligned} \text{minimize} \quad & \sum_{v\in V} y_v & \\ \text{subject to} \quad & A^T y \geq 1 & (\forall e \in E) \\ & y \geq 0 & (\forall v \in V) \end{aligned}$$

where incidence matrix A is totally unimodular. So we can rewrite Primal LP of maximum matching:

$$\begin{aligned} \text{maximize} \quad & \sum_{e\in E} x_e & \\ \text{subject to} \quad & Ax \leq 1 & (\forall v \in V) \\ & x \geq 0 & (\forall e \in E) \end{aligned}$$

1) Since A is totally unimodular, all poles of a polyhedron $\{x : Ax \leq 1, x \geq 0\}$ are integer points $\implies$ There exists an integer optimal solution, corresponding exactly to a matching.

2) Since A is totally unimodular, then A^T is totally unimodular. By similar analysis, the dual LP also has an integer optimal solution. $\implies$ exists an optimal solution with a 0/1 integer, corresponding exactly to a vertex cover.

By 1), 2), and Strong Dual LP Theorem, Primal LP optimum=Dual LP optimum,
i.e. maximum matching size = minimum vertex cover size. □

(e)

Consider a graph with triangle structure:
$G = (V, E)$, where $V = \{1, 2, 3\}$, $E = \{(1, 2), (2, 3), (3, 1)\}$.
Apparently, it is not bipartite.

- Maximum matching: 1
- Minimum vertex cover: 2

$\implies 1 \neq 2$, the theorem does not hold in non-bipartite graphs.

Problem 4

I did this assignment for 5 hours. Difficulty Score: 5. I did it by myself.